there's no sequenced-before (and thus no happens-before) relationship between them. Of course, all operations in a statement happen before all of the operations in the next statement.

This is really just a restatement of the single-threaded sequencing rules you're used to, so what's new? The new part is the interaction between threads: if operation A on one thread inter-thread happens-before operation B on another thread, then A happens-before B. This doesn't really help much: you've just added a new relationship (inter-thread happens-before), but this is an important relationship when you're writing multithreaded code.

At the basic level, inter-thread happens-before is relatively simple and relies on the synchronizes-with relationship introduced in section 5.3.1: if operation A in one thread synchronizes-with operation B in another thread, then A inter-thread happens-before B. It's also a transitive relation: if A inter-thread happens-before B and B inter-thread happens-before C, then A inter-thread happens-before C. You saw this in listing 5.2 as well.

Inter-thread happens-before also combines with the sequenced-before relation: if operation A is sequenced before operation B, and operation B inter-thread happens-before operation C, then A inter-thread happens-before C. Similarly, if A synchronizes-with B and B is sequenced before C, then A inter-thread happens-before C. These two together mean that if you make a series of changes to data in a single thread, you need only one synchronizes-with relationship for the data to be visible to subsequent operations on the thread that executed C.

These are the crucial rules that enforce ordering of operations between threads and make everything in listing 5.2 work. There are some additional nuances with data dependency, as you'll see shortly. In order for you to understand this, I need to cover the memory-ordering tags used for atomic operations and how they relate to the synchronizes-with relation.

5.3.3. Memory ordering for atomic operations

There are six memory ordering options that can be applied to operations on atomic types: `memory_order_relaxed`, `memory_order_consume`, `memory_order_acquire`, `memory_order_release`, `memory_order_acq_rel`, and `memory_order_seq_cst`. Unless you specify otherwise for a particular operation, the memory-ordering option for all operations on atomic types is `memory_order_seq_cst`, which is the most stringent of the available options. Although there are six ordering options, they represent three models: *sequentially consistent* ordering (`memory_order_seq_cst`), *acquire-release* ordering (`memory_order_consume`, `memory_order_acquire`, `memory_order_release`, and `memory_order_acq_rel`), and *relaxed* ordering (`memory_order_relaxed`).

These distinct memory-ordering models can have varying costs on different CPU architectures. For example, on systems based on architectures with fine control over the visibility of operations by processors other than the one that made the change, additional synchronization instructions can be required for sequentially consistent ordering over acquire-release ordering or relaxed ordering and for acquire-release ordering over relaxed ordering. If these systems have many processors, these additional synchronization instructions may take a significant amount of time,